KernelMix reference manual

Nilson Chapagain

Contents

1 KernelMix **1**
1.1 Install . . . 2
1.2 Example . . . 2
1.3 Documentation . . . 2
1.4 References and license . . . 3

2 Model and interpretation **3**
2.1 Continuous outcomes . . . 3
2.2 Binary outcomes . . . 3
2.3 Priors and exposure selection . . . 4
2.4 Inputs and computation . . . 4
2.5 Prediction and contrasts . . . 4
2.6 Diagnostics and scope . . . 5

3 An exposure-mixture analysis **5**
3.1 Define the data . . . 5
3.2 Fit and inspect a continuous outcome . . . 5
3.3 Predict at exposure profiles . . . 6
3.4 Summarize exposure associations . . . 6
3.5 Fit a binary outcome . . . 6
3.6 Plot the results . . . 7
3.7 Save and load a fit . . . 7

4 API reference **7**
4.1 Fitting . . . 8
4.2 Fit results and diagnostics . . . 9
4.3 Prediction . . . 9
4.4 Exposure contrasts and curves . . . 10
4.5 Plotting . . . 11
4.6 Kernel helper . . . 11
4.7 Method sources . . . 12

5 Checks and development **12**
5.1 Run checks . . . 12
5.2 Numerical evidence . . . 12
5.3 Build the manual . . . 13

1 KernelMix

Bayesian kernel machine regression (BKMR) for exposure mixtures in Python. KernelMix estimates a flexible exposure–response function, with linear adjustment for covariates. It supports continuous outcomes and binary outcomes with a probit link. NumPy and SciPy handle fitting and prediction; R is not required.

Analysis	Functions
Fit continuous or binary outcomes	`fit`, `Prior`
Select exposures or exposure groups	`selection="component"` or `"group"`, posterior inclusion probabilities
Inspect results	`summary`, `diagnostics`, `posterior_samples`
Predict	`predict_h`, `predict`, `posterior_moments_h`
Describe exposure associations	Overall, single-exposure and interaction contrasts; response curves and surfaces
Plot	Chains, inclusion probabilities, curves, contrasts and surfaces

1.1 Install

With access to this repository:

```
python -m pip install "kernelmix[plots] @ git+https://github.com/nilson01/kernelmix.git"
```

Or, from a downloaded source folder:

```
python -m pip install ".[plots]"
```

Python 3.10 or later is required. Omit `[plots]` if plots are not needed.

1.2 Example

```
import numpy as np
from kernelmix import fit, univariate_response, plot_response

rng = np.random.default_rng(17)
Z = rng.normal(size=(24, 2))
y = np.sin(Z[:, 0]) + 0.3 * Z[:, 1] + rng.normal(0, 0.5, 24)
result = fit(y, Z, draws=200, warmup=200, chains=2, seed=17)
print(result.summary())
curve = univariate_response(result, variable=0, points=25, seed=17)
ax = plot_response(curve)
```

Example output (first rows):

```
BKMRFit(family='gaussian', n=24, exposures=2, chains=2, draws=200)
95% pointwise credible intervals
Parameter             Mean         SD      Lower      Upper
lambda               16.93      9.047      4.145      38.61
sigma2              0.2159    0.08346     0.1142     0.4188
```

These short chains trigger mixing warnings. This example demonstrates the workflow. Choose chain lengths using mixing, effective sample size and the precision needed for the analysis.

The curve holds the second exposure at its sample median and subtracts the all-median reference. It shows an association, not an identified causal effect.

1.3 Documentation

- Tutorial: data, fitting, prediction and plots.
- API: grouped functions, inputs and returned values.
- Model and interpretation: equations, priors and limits.
- Reference manual: printable documentation.
- Checks and development: tests and installed-R comparisons.

Dense covariance calculations limit practical sample size. Only the Gaussian kernel is supported. Group selection allows at most one selected exposure per group. Diagnostics cannot establish convergence or validate model assumptions.

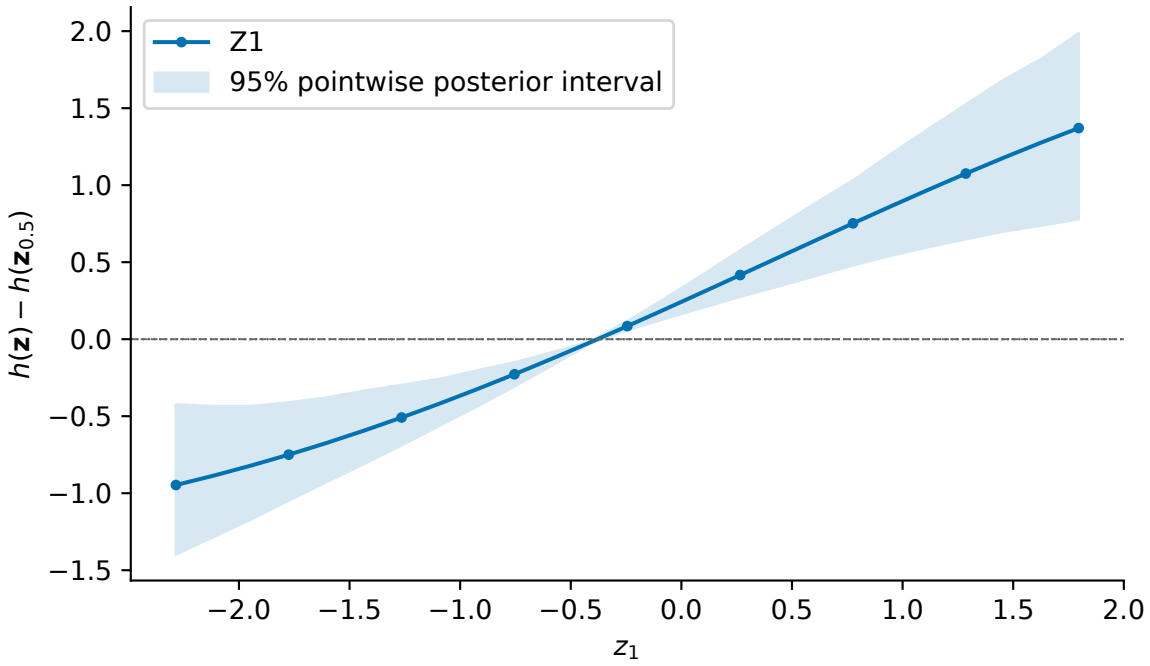


Figure 1: Exposure response curve with pointwise 95% credible intervals

1.4 References and license

The method is due to Bobb et al. (2015). The software and binary-outcome extension are described by Bobb et al. (2018). KernelMix is a separate Python implementation. Selected calculations are checked against R `bkmr` 0.2.2. See NOTICE for attribution and LICENSE for GPL-2.0-only terms.

2 Model and interpretation

2.1 Continuous outcomes

Following Bobb et al. (2015),

$$Y_i = h(\mathbf{z}_i) + \mathbf{x}_i^{\mathsf{T}}\boldsymbol{\beta} + \epsilon_i, \qquad \epsilon_i \overset{\text{iid}}{\sim} N(0, \sigma^2).$$

Here, Y_i is the outcome, $\mathbf{z}_i$ contains the M exposures, $\mathbf{x}_i$ contains adjustment covariates, and $\boldsymbol{\beta}$ contains their coefficients. The function h allows nonlinear exposure associations and interactions. Its prior at the observed exposure profiles is

$$\mathbf{h} = (h_1, \ldots, h_n)^{\mathsf{T}} \sim N(\mathbf{0}, \tau \mathbf{K}_{Z,\mathbf{r}}), \qquad K(\mathbf{z}, \mathbf{z}'; \mathbf{r}) = \exp\left\{ - \sum_{m=1}^{M} r_m (z_m - z'_m)^2 \right\}.$$

The relevance parameters $r_m \geq 0$ determine how each exposure changes the kernel. An exposure with $r_m = 0$ does not affect h. In the software, `lambda` is $\lambda = \tau / \sigma^2$. Integrating out $\mathbf{h}$ gives covariance $\sigma^2(\mathbf{I} + \lambda \mathbf{K}_{Z,\mathbf{r}})$.

2.2 Binary outcomes

With `family="binomial"`, the model uses a probit link:

$$\Pr(Y_i = 1 \mid h, \boldsymbol{\beta}) = \Phi\{h(\mathbf{z}_i) + \mathbf{x}_i^{\mathsf{T}}\boldsymbol{\beta}\}.$$

Here Φ is the standard normal cumulative distribution function. Residual variance on the latent normal scale is fixed at one. This extension follows Bobb et al. (2018). Probability contrasts depend on both exposure profiles and adjustment covariates. They are not odds ratios. Separated linear adjustment designs are rejected: the flat coefficient prior can give an improper posterior in this case.

2.3 Priors and exposure selection

`Prior()` defaults match R `bkmr` 0.2.2. Gamma distributions below use shape and **rate**. The inverse-gamma density is proportional to $(\sigma^2)^{-a-1}\exp(-b/\sigma^2)$ for shape a and parameter b.

Quantity	Default
λ	Gamma(1, 0.1)
σ^2, continuous outcome	Inverse-gamma(0.001, 0.001)
Active r_m	$1/r_m \sim$ Uniform$(0, 100)$
Inclusion probability π	Beta(1,1)
$\boldsymbol{\beta}$	Flat prior

The inverse-uniform relevance prior has an unbounded upper tail. A finite-chain average of r_m does not imply that its posterior expectation exists. Inspect quantiles and mixing; assess prior sensitivity for the application. `Prior(r_distribution="gamma")` and `"unif"` provide other positive priors. The gamma parameters use mean and standard deviation. Uniform bounds apply to r_m for `"unif"` and to $1/r_m$ for `"invunif"`.

With component selection, $\delta_m \sim$ Bernoulli(π) indicates whether $r_m > 0$. Excluded exposures have $r_m = 0$. Group selection uses the groups S_g and indicator ω_g from Bobb et al. (2015): each active group contains **one** selected exposure, chosen uniformly within that group. KernelMix does not offer unequal within-group prior probabilities. A group PIP is its posterior inclusion probability; a conditional PIP is an exposure's inclusion probability given that its group is active. A never-selected group has an undefined conditional PIP. PIPs depend on the model and priors; they do not measure causal importance.

2.4 Inputs and computation

`fit` adds an intercept and standardizes exposures by default. Standardization uses the retained training mean and sample standard deviation. Prediction reuses these values. Pass `intercept=False` or `standardize=False` to disable either. Do not include a constant adjustment column when adding an intercept. R `bkmr` does not automatically apply these Python preprocessing defaults.

`missing="raise"` rejects missing data. `missing="drop"` removes rows jointly across the outcome, exposures and covariates. Retained original row indices are stored in `result.data["rows"]`. Complete-case fitting does not correct bias caused by missing data. Covariates must be numeric; categorical encoding is the caller's responsibility.

Chains run sequentially with separate random streams. `draws` counts saved draws per chain after `warmup`; `thin` controls the spacing of saved states. Proposals are fixed during sampling. `sampling_info` records acceptance counts and runtime. The Gaussian model integrates out h during parameter sampling. The binary model also samples correlated, sign-constrained latent outcomes. Predictions condition on the saved latent outcomes for binary fits.

Dense matrix factorization requires roughly cubic work in n per parameter update. Working-memory checks reject oversized fitting or prediction requests; they are protective estimates, not a promise about peak process memory.

2.5 Prediction and contrasts

`predict_h` draws h jointly over requested profiles. `predict(kind="mean")` returns outcome means, or probabilities for binary outcomes. `predict(kind="observation")` also includes new observation variation. The latent binary scale is the underlying normal scale.

Contrasts subtract **joint** posterior draws so that covariance is retained. `overall_effect` compares all-exposure quantile profiles with a reference profile. `single_exposure_effects` changes one exposure while holding the

others fixed. `interaction_effects` returns a difference of two single-exposure contrasts. Curves and surfaces vary one or two exposures; others are held fixed. Their intervals are equal-tailed, pointwise credible intervals, not simultaneous bands. On the response scale, adjustment covariates default to their retained training means unless `X` supplies a different profile.

Marginal quantile profiles may be uncommon in correlated data. Check exposure support before interpreting these contrasts. Causal interpretation requires additional identification assumptions; fitting a model does not supply them.

2.6 Diagnostics and scope

`diagnostics()` reports rank-normalized/folded split $\widehat{R}$, bulk effective sample size and tail effective sample size. Large $\widehat{R}$, small effective sample sizes, or constant sampled traces need investigation. Fixed parameters are excluded from these diagnostics. Trace plots and sensitivity checks remain necessary; favorable diagnostics do not prove convergence.

KernelMix supports the dense Gaussian kernel with continuous and binary outcomes. It does not implement random intercepts, clustered outcomes, low-rank approximations, count or survival outcomes. Numerical comparisons cover selected fixed states and small analytic posterior targets. They do not establish general frequentist coverage or reproduce the original paper's studies.

3 An exposure-mixture analysis

KernelMix fits Bayesian kernel machine regression (BKMR) for a continuous outcome or a binary outcome with a probit link. The exposure-response function, called `h`, can include nonlinear associations and interactions. Adjustment covariates enter linearly.

This tutorial uses small, deterministic example data. Run its Python blocks in order after installing the source package as described in the README. The short chains make the examples quick to run. They are too short to support inference. For an analysis, choose longer runs, inspect mixing and effective sample sizes, and check sensitivity to prior choices.

3.1 Define the data

Each row is an observation. Columns of `Z` are exposures; columns of `X` are adjustment covariates. Keep `Z` in its original units. The fit stores the training means and sample standard deviations and reuses them for prediction.

```
import numpy as np
import kernelmix as km

t = np.linspace(-1.0, 1.0, 12)
Z = np.column_stack((1.5 + t, 2.0 + np.sin(3 * t), 10.0 + np.cos(5 * t)))
X = t[:, None]
y = 0.3 + np.sin(2 * t) + 0.2 * t**2
names = ["Exposure A", "Exposure B", "Exposure C"]
```

The default adds an intercept, so `X` should not include a constant column. Missing values raise an error. Use `missing="drop"` only when joint complete-case analysis is appropriate; that option drops a row if its outcome, exposure or adjustment value is missing.

3.2 Fit and inspect a continuous outcome

Component selection allows each exposure to enter or leave the model. Its posterior inclusion probability (PIP) is the proportion of saved states in which that exposure is included.

```
gaussian = km.fit(
    y, Z, X,
    family="gaussian", selection="component",
    exposure_names=names, covariate_names=["Adjustment"],
    draws=12, warmup=12, chains=2, seed=19,
```

```
)
print(gaussian.summary())
parameter_rows = gaussian.summary_table()
pip_rows = gaussian.inclusion_probabilities()
checks = gaussian.diagnostics()
```

The summary gives posterior means, standard deviations and pointwise credible intervals. Diagnostics include split R-hat, which compares chains, and effective sample sizes, which account for dependence between saved draws. These checks help detect sampling problems; passing them does not establish that the model fits the scientific question. PIPs depend on the selection model and priors and are not causal importance scores.

3.3 Predict at exposure profiles

Supply new exposures in the training column order and their original units. New adjustment rows contain the raw `X` columns, without an intercept. They are required when adjustments were fitted.

```
Znew = Z[:3].copy()
Xnew = X[:3].copy()
mean_prediction = gaussian.predict(Znew, Xnew, seed=20)
observation_prediction = gaussian.predict(
    Znew, Xnew, kind="observation", seed=20,
)
h_prediction = gaussian.predict_h(Znew, seed=20)
prediction_rows = mean_prediction.to_records()
```

The default predicts outcome means. Observation predictions add residual noise and describe new outcomes. `predict_h` returns only the exposure function, without the intercept or adjustment contribution. Prediction objects retain joint draws, pointwise intervals and covariance between prediction rows. Calling `predict()` without new data uses the retained training rows and their adjustments.

3.4 Summarize exposure associations

Overall contrasts compare profiles at which every exposure is at a specified marginal quantile. Single-exposure contrasts change one exposure while fixing the others. These are conditional associations; a causal interpretation needs additional scientific assumptions.

```
overall = km.overall_effect(gaussian, reference=0.5, seed=21)
single = km.single_exposure_effects(gaussian, seed=21)
interaction = km.interaction_effects(
    gaussian, variables=["Exposure A", "Exposure B"], seed=21,
)
curve = km.univariate_response(
    gaussian, variable="Exposure A", points=7, seed=21,
)
surface = km.bivariate_response(
    gaussian, variables=("Exposure A", "Exposure B"), points=4, seed=21,
)
overall_rows = overall.to_records()
```

By default, these summaries describe changes in `h`. Curves and surfaces hold other exposures at their medians and subtract a common all-median reference profile. A centered value of zero means no change from that reference. `center=False` returns the uncentered curve or surface. Interval limits are pointwise posterior quantiles, not a simultaneous band across the whole curve.

Each contrast uses joint prediction draws for the profiles being compared. This retains their covariance. A profile compared with itself has exactly zero contrast. Marginal quantile combinations can be poorly represented among observed profiles, especially with correlated exposures; examine data support when interpreting them.

3.5 Fit a binary outcome

For a binary outcome, use `family="binomial"`. The probit link maps the latent mean to a probability with the standard normal cumulative distribution function. Outcomes must be zero or one.

```
y_binary = np.array([0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1])
binary = km.fit(
    y_binary, Z, X,
    family="binomial", exposure_names=names,
    covariate_names=["Adjustment"],
    draws=12, warmup=12, chains=2, seed=22,
)
probability_prediction = binary.predict(Znew, Xnew, seed=23)
probability_contrast = km.overall_effect(
    binary, scale="response", X=[0.0], seed=23,
)
```

Binary mean predictions are probabilities. With `kind="observation"`, predictions are zero-or-one outcomes. A response-scale contrast subtracts predicted probabilities after applying the probit transformation to each joint draw. It is not an odds ratio. In exposure summaries, `X` is one raw adjustment vector held constant at every profile. If it is omitted, response-scale summaries use the retained training covariate means. Latent-scale summaries use `h` alone and do not accept `X`.

3.6 Plot the results

Install the optional Matplotlib dependency to use these functions. Each returns an axes object. The caller controls display and export.

```
trace_ax = km.plot_trace(gaussian)
pip_ax = km.plot_pips(gaussian)
curve_ax = km.plot_response(curve)
overall_ax = km.plot_overall(overall)
surface_ax = km.plot_surface(surface)

# Optional: display the figures in an interactive session.
# import matplotlib.pyplot as plt
# plt.show()
```

Curve and overall plots include pointwise posterior intervals. The surface plot displays the posterior mean only; its result object also retains lower and upper limits. Plot exposures separately when their units differ. The trace excludes warmup draws.

3.7 Save and load a fit

A fit includes its retained training data as well as saved states. Store and share it with the same care as the input data. The archive uses a non-pickle format.

```
from pathlib import Path
from tempfile import TemporaryDirectory

with TemporaryDirectory() as directory:
    path = Path(directory) / "mixture-fit.npz"
    gaussian.save(path)
    restored = km.BKMRFit.load(path)
    restored_prediction = restored.predict(Znew, Xnew, seed=20)
```

See the API reference for argument defaults and result fields. The method is described by Bobb et al. (2015); Bobb et al. (2018) describes exposure summaries and the probit extension. KernelMix's pairwise interaction summary and median-reference curve centering are specified in the API reference.

4 API reference

Import the public API with `import kernelmix as km`. Arrays are NumPy arrays; pandas is not required. Examples that connect these functions are in the tutorial.

4.1 Fitting

```
km.fit(
    y, Z, X=None, *, family="gaussian", draws=1000, warmup=1000,
    chains=4, thin=1, seed=None, standardize=True, intercept=True,
    selection="none", groups=None, prior=None, r_fixed=None, missing="raise",
    exposure_names=None, covariate_names=None, initial=None, proposal_scales=None,
)
```

Returns `BKMRFit`.

Input	Meaning
`y`, `Z`, `X`	Outcome vector, exposure matrix and optional adjustment matrix, with matching rows. Supply raw adjustment columns without an intercept when `intercept=True`.
`family`	`"gaussian"` for continuous outcomes or `"binomial"` for binary probit regression. Binary residual variance is fixed at one.
`draws`, `warmup`, `chains`, `thin`	Retained states per chain, discarded iterations, sequential chains and spacing between retained states. Total iterations per chain are `warmup + draws * thin`. At least two retained states are required.
`seed`	Nonnegative integer or `None`; uses local random generators without changing NumPy's global random state.
`standardize`	Center exposures and divide by retained-training sample standard deviations. Constant exposures are invalid when this is enabled.
`selection`	`"none"`, `"component"` or `"group"`. Group selection requires one `groups` label per exposure and permits at most one active exposure in each included group.
`prior`	`Prior`, a dictionary of its fields, or `None` for defaults.
`r_fixed`	Finite nonnegative vector fixing exposure relevance values; disallows variable selection.
`missing`	`"raise"` or joint complete-case `"drop"`. Infinity remains invalid.
Names	Optional unique names for exposure and raw adjustment columns. Otherwise use input DataFrame columns, or generated names.
`initial`	Shared dictionary of initial `beta`, `r`, `lambda`, `sigma2`, `delta`, `pi` and, for binary outcomes, `ystar`. Omitted entries receive defaults.
`proposal_scales`	Dictionary containing positive log-scale proposal standard deviations for `"r"` and/or `"lambda"`; both default to `0.3` and stay fixed throughout sampling.

The adjustment design must have full column rank and fewer columns than retained observations. Binary adjustment designs with complete or quasi-complete separation are rejected under the flat coefficient prior. Fitting uses exact dense covariance matrices, with quadratic memory and cubic linear-algebra cost in the number of observations. Successful fitting does not establish convergence.

4.1.1 Prior configuration

```
km.Prior(
    r_distribution="invunif", r_mean=5.0, r_sd=5.0,
    r_lower=0.0, r_upper=100.0, lambda_mean=10.0, lambda_sd=10.0,
```

```
    sigma2_shape=0.001, sigma2_rate=0.001,
    inclusion_alpha=1.0, inclusion_beta=1.0,
)
```

`r_distribution` accepts `"gamma"`, `"unif"` and `"invunif"`. The inverse-uniform option places the uniform distribution on `1/r`, so its bounds apply to the reciprocal of an active relevance value. Excluded exposures have relevance zero. Gamma priors use mean and standard deviation; the residual variance uses inverse-gamma shape and rate. Inclusion probabilities use a beta prior. The residual variance prior is used for Gaussian outcomes.

4.2 Fit results and diagnostics

`BKMRFit` retains `data`, `samples`, `settings` and `sampling_info`. Properties `family`, `exposure_names` and `covariate_names` describe the fitted inputs. Stored arrays are copied and read-only; extracted samples are independent copies.

Method	Return and use
`fit.summary(probability=0.95)`	Text summary of parameter means, SDs, equal-tailed intervals, available PIPs and diagnostic messages.
`fit.summary_table(probability=0.95)`	List of dictionaries with `parameter`, `mean`, `sd`, `lower`, `upper`.
`fit.posterior_samples(parameter=None, combine_chains=False)`	Dictionary of arrays, or one named parameter array. Leading axes are chain and retained draw; combining chains merges those axes.
`fit.inclusion_probabilities()`	Exposure names and PIPs for selection fits. Group fits also return group PIP and within-group conditional PIP; the conditional value is undefined (`NaN`) if a group was never selected. Raises without selection.
`fit.diagnostics()`	Parameter diagnostics and messages. Uses rank-normalized/folded split R-hat and bulk/tail effective sample sizes.
`fit.save(path)`	Save training data, settings and states in a compressed non-pickle archive. Replaces the destination atomically.
`km.BKMRFit.load(path, *, max_bytes=512 * 1024 * 1024)`	Load and validate an archive, limiting total uncompressed member size.
`km.diagnose(values)`	Diagnostics for one finite scalar-parameter array shaped `(chains, draws)`.

Saved parameters include `beta`, `r`, `lambda`, `sigma2`, `delta` and `pi`; binary fits also retain `ystar`. `delta` records exposure inclusion. `beta` includes the added intercept when fitted. No-selection fits store `pi=1`. Diagnostics cannot establish model validity; R-hat requires multiple chains and short or constant sequences can yield undefined diagnostics.

4.3 Prediction

```
fit.predict_h(Znew=None, *, probability=0.95, max_draws=None, seed=None)
fit.predict(
    Znew=None, Xnew=None, *, kind="mean", scale="response",
    probability=0.95, max_draws=None, seed=None,
)
fit.posterior_moments_h(Znew=None, max_draws=None)
```

All new exposure values use original units and the training column order. Named input columns must match the stored names and order. Training standardization is reused. `Xnew` contains raw adjustment columns without the intercept; it is required for new profiles when adjustments were fitted. With no new data, retained training rows and their adjustments are used.

Call	Quantity
`predict_h`	Joint draws of `h`, excluding intercept and adjustment terms.
`predict(..., kind="mean", scale="response")`	Gaussian outcome means or binary probabilities.
`predict(..., kind="observation", scale="response")`	Gaussian future outcomes including residual noise, or binary zero-or-one outcomes.
`predict(..., kind="mean", scale="latent")`	`h` plus the linear adjustment contribution, before the binary probit transformation.
`predict(..., kind="observation", scale="latent")`	Latent outcomes including Gaussian residual noise; binary residual variance is one.
`posterior_moments_h`	Dictionary with deterministic integrated `mean` and `covariance` for `h`, averaging conditional moments over saved states.

`max_draws` selects at most that many evenly spaced saved states **per chain**, with a minimum of two. It does not refit. `seed` controls conditional prediction draws. A `Prediction` contains `draws` shaped `(chains, selected_draws, rows)`, `mean`, `sd`, `lower`, `upper`, joint `covariance`, `probability`, `scale` and `kind`. Its `to_records()` method returns one summary dictionary per row. Intervals are equal-tailed pointwise posterior intervals; observation intervals also include observation noise.

4.4 Exposure contrasts and curves

```
km.overall_effect(
    fit, quantiles=(0.25, 0.5, 0.75), reference=0.5, scale="latent",
    X=None, probability=0.95, max_draws=None, seed=None,
)
km.univariate_response(
    fit, variable=None, grid=None, points=50, fixed_quantile=0.5,
    center=True, scale="latent", X=None, probability=0.95,
    max_draws=None, seed=None,
)
km.bivariate_response(
    fit, variables=(0, 1), grids=None, points=20, fixed_quantile=0.5,
    center=True, scale="latent", X=None, probability=0.95,
    max_draws=None, seed=None,
)
km.single_exposure_effects(
    fit, quantiles=(0.25, 0.75), fixed_quantiles=(0.25, 0.5, 0.75),
    scale="latent", X=None, probability=0.95, max_draws=None, seed=None,
)
km.interaction_effects(
    fit, variables=None, quantiles=(0.25, 0.75), fixed_quantile=0.5,
    scale="latent", X=None, probability=0.95, max_draws=None, seed=None,
)
```

Function	Quantity and result
`overall_effect`	Each all-exposure quantile profile minus the all-exposure `reference` profile. Returns `EffectResult`.
`univariate_response`	One named or zero-based indexed exposure, or all exposures with `variable=None`. Other exposures stay at `fixed_quantile`. Returns one joint `CurveResult`.
`bivariate_response`	A Cartesian grid for two distinct exposures. Other exposures stay at `fixed_quantile`. Returns `SurfaceResult`.

Function	Quantity and result
`single_exposure_effects`	High-minus-low quantile contrast for each exposure at every background `fixed_quantiles` value. Returns `EffectResult`.
`interaction_effects`	All unordered pairs from the selected names/indices, or all exposures by default. Returns `EffectResult`. For each pair the contrast is high/high - low/high - high/low + low/low, with other exposures fixed.

Quantiles come from retained original training exposures and must lie in `[0, 1]`. Default grids span the observed range. Explicit grids must be finite, nonempty and strictly increasing. For an all-variable univariate call, a supplied `grid` is shared across exposures. `grids` contains one sequence for each surface axis. Calls are limited to 4096 output locations because joint prediction and contrast matrices are dense.

With `center=True`, curves and surfaces subtract the **all-median** reference profile, even if `fixed_quantile` is changed. This differs from R `bkmr` curve centering by the average fitted grid value. The pairwise interaction above also differs from the R summary that changes all background exposures together. These definitions should be specified when comparing results between packages.

Latent summaries use `h`. Response summaries use Gaussian means or binary probabilities and hold `X` constant. `X` is one raw adjustment vector; when omitted it defaults to retained training covariate means. `X` is rejected for latent summaries. Binary probabilities are transformed before contrasts are calculated. All contrasts use joint draws to preserve covariance and exactly cancel identical profiles.

Result objects contain `draws` with axes `(chains, selected_draws, locations)`, `mean`, `sd`, `lower`, `upper`, `locations`, `profiles`, `coefficients`, `scale`, `probability`, `kind` and `metadata`. `locations` identifies each variable, pair, quantile or grid value. `profiles` stores distinct original-unit prediction rows; `coefficients` maps their joint draws to reported summaries. `to_records()` returns location fields and numeric summaries. A `SurfaceResult` also contains `variables`, `grids` and `shape`; reshape its flat summaries to `shape`, with the second grid changing fastest.

These summaries describe conditional associations. Marginal quantile profiles can combine values rarely observed together. Credible intervals do not address poor exposure support, unmeasured confounding or model misspecification.

4.5 Plotting

Matplotlib is an optional dependency and is imported only when a plot is requested.

```
km.plot_trace(fit, parameter="lambda", indices=None, ax=None)
km.plot_pips(fit, ax=None)
km.plot_response(curve, ax=None)
km.plot_overall(effect, ax=None)
km.plot_surface(surface, ax=None)
```

Each function returns a Matplotlib axes object, creates one when `ax=None`, and leaves display, saving and global styles to the caller. `indices` selects a scalar component for a vector-valued trace parameter, for example `indices=0` for the first `beta` coefficient. Traces contain retained draws only. PIP plots require a selection fit.

`plot_response` accepts a univariate curve; `plot_overall` accepts an overall effect; `plot_surface` accepts a bivariate surface with at least two values on each grid. Curve and overall plots show pointwise posterior intervals. A surface shows the posterior mean only, with zero-centered colors when centered. Result objects retain its uncertainty. Request separate curves for exposures with different units.

4.6 Kernel helper

```
km.gaussian_kernel(Z1, Z2=None, r=None)
```

Returns a Gaussian kernel matrix between the supplied exposure rows. `Z2=None` uses `Z1` for both sides; `r=None` uses unit relevance for each column. Relevance values are nonnegative. This helper uses the values supplied directly and does not apply a fit's training transform.

4.7 Method sources

- Bobb et al. (2015), Bayesian kernel machine regression for estimating the health effects of multi-pollutant mixtures.
- Bobb et al. (2018), Statistical software for analyzing the health effects of multiple concurrent exposures via Bayesian kernel machine regression.

5 Checks and development

5.1 Run checks

```
python -m pip install -e ".[dev]"
python -m pytest
python -m ruff check src tests tools
python tools/check_public.py --history
python -m build
python -m twine check dist/*
```

CI builds a wheel and tests it outside the source checkout on Linux with Python 3.10 and 3.14, and Windows with Python 3.13. The minimum-dependency job pins NumPy 1.24.0, SciPy 1.10.0 and Matplotlib 3.7.0. See the repository's Actions results for the outcome on each commit; configuration alone does not establish that a platform passed.

5.2 Numerical evidence

Check	Purpose and limit
Independent kernel and conditional-normal calculations	Check covariance, prediction and joint contrasts
Installed R `bkmr` 0.2.2	Compare constructed saved states for Gaussian and probit models, exact conditional moments, PIPs and prior densities
Integrated Gaussian posterior	Compare sampled amplitude quantiles with one-dimensional quadrature after integrating out residual variance
Probit orthant posterior	Compare sampled amplitude quantiles with the analytic two-observation normal orthant likelihood
Uninformative selection case	Check the beta-binomial count distribution when exposure selection cannot change the kernel
Diagnostic examples	Detect shifted chains, autocorrelation and constant traces
Package and input checks	Exercise saved objects, preprocessing, errors, distribution contents and installed examples

The posterior-target tests use fixed seeds and Monte Carlo tolerances based on effective sample size. They are small correctness checks, not a repeated-dataset performance study. No general coverage, paper-study reproduction or complete R API equivalence is claimed.

5.2.1 Reproduce the R fixtures

R is needed only to regenerate fixtures. Use an isolated R library with `bkmr` **0.2.2**, `jsonlite` and the dependencies declared by that version of `bkmr` (including `fields`, `truncnorm` and `tmvtnorm`). Its CRAN archive has SHA256 `7ed29f74c4c78f3b8c663eac5ec417b4ea9bc586d789e8049e3ddc51541af9af`. Set `R_LIBS` to that library, then run:

```
Rscript tools/export_reference.R tests/fixtures/r_reference.json
python -m pytest tests/test_reference.py
```

The exporter verifies the installed version and records R/package versions. It calls installed `ComputePostmeanHnew` and `ExtractPIPs`, plus internal kernel and prior helpers. The fixtures contain **constructed states, not fitted R MCMC chains**. Preprocessing and intercept columns are supplied explicitly.

The elementwise tolerance is `rtol=2e-11, atol=2e-12`. R's exact summary uses sample covariance between conditional means (denominator $S - 1$); Python's finite empirical mixture uses denominator S, where S is the retained state count. The comparison adjusts this convention explicitly. Sampling proposals and random-number streams differ; equal seeds do not produce matching chains.

5.3 Build the manual

The editable sources are the README and Markdown pages in `docs`. Install the documentation extra and a local `pdflatex` distribution:

```
python -m pip install ".[docs]"
python tools/build_manual.py
```

Pandoc and LaTeX build the PDF from Markdown. Intermediate TeX stays in a temporary directory. The small README example generates the response figure; its plotted values are also available in `docs/figures/response.csv`. The figure shows pointwise credible limits from short illustrative chains, not a performance benchmark.